



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Aprendizaje Automático Probabilístico

Probabilidad: modelos univariados (actividades)

Alfons Juan

DSIC

Departamento de Sistemas
Informáticos y Computación

Índice

2.1	Introducción	1
2.2	Variables aleatorias	3
2.2.1	Variables aleatorias discretas	3
2.2.2	Variables aleatorias continuas	4
2.2.3	Conjuntos de variables aleatorias relacionadas	5
2.2.4	Independencia e independencia condicional	6
2.2.5	Momentos de una distribución	7
2.3	Regla de Bayes	8
2.4	Distribuciones Bernoulli y binomial	9
2.5	Distribuciones categórica y multinomial	17
2.6	Distribución (normal) Gaussiana univariada	22

2.1. Introducció

- Considera el lanzamiento de una moneda justa, esto es, con idéntica probabilidad de obtener cara y cruz. La incertidumbre sobre el resultado es de tipo:

1. Epistémica o de modelo

2. Aleatoria o de datos

3. De ambos tipos

4. De ninguno

► Supón que:

▷ X e Y se escogen uniformemente al azar de $\mathcal{X} = \{1, 2, 3, 4\}$

▷ A es el evento $X \in \{1, 2, 3\}$

▷ B es el evento $Y \in \{3, 4\}$

Determina la probabilidad conjunta $\Pr(A, B)$

2.2. Variables aleatorias

2.2.1. Variables aleatorias discretas

- Sea X una variable aleatoria discreta de espacio muestral $\mathcal{X}$. Sea $p : \mathcal{X} \rightarrow \mathbb{R}$ una función sobre la consideramos dos condiciones:

$$0 \leq p(x) \leq 1 \quad \text{para todo } x \in \mathcal{X}$$

$$\sum_{x \in \mathcal{X}} p(x) = 1$$

Decimos que p es función (de masa) de probabilidad (pmf) sii:

1. Cumple la primera condición
2. Cumple la segunda condición
- 3.** Cumple ambas condiciones
4. En cualquier caso

2.2.2. Variables aleatorias continuas

- Sea X una variable aleatoria continua de espacio muestral $\mathcal{X}$. Sea $p : \mathcal{X} \rightarrow \mathbb{R}$ una función sobre la consideramos dos condiciones:

$$0 \leq p(x) \quad \text{para todo } x \in \mathcal{X}$$

$$\int_{x \in \mathcal{X}} p(x) dx = 1$$

En general, p es función de densidad (de probabilidad) (pdf) sii:

1. Cumple la primera condición
2. Cumple la segunda condición
- 3.** Cumple ambas condiciones
4. En cualquier caso

2.2.3. Conjuntos de variables aleatorias relacionadas

- Sean X e Y variables aleatorias binarias de distribución conjunta:

$p(X, Y)$	$Y = 0$	$Y = 1$
$X = 0$	0.1	0.4
$X = 1$	0.4	0.1

Halla las marginales y la condicional de Y dada X

2.2.4. Independencia e independencia condicional

- Sean X e Y variables aleatorias binarias de distribución conjunta:

$p(X, Y)$	$Y = 0$	$Y = 1$
$X = 0$	0.1	0.4
$X = 1$	0.4	0.1

¿Son independientes?

2.2.5. Momentos de una distribución

- Sea X una variable aleatoria discreta de pmf:

k	0	1	2	3	> 3
$p(X = k)$	0.1	0.4	0.3	0.2	0.0

Halla $\mathbb{E}[X]$, $\mathbb{V}[X]$ y $\mathbb{E}[Y]$ con $Y = (X - 2)^2$

2.3. Regla de Bayes

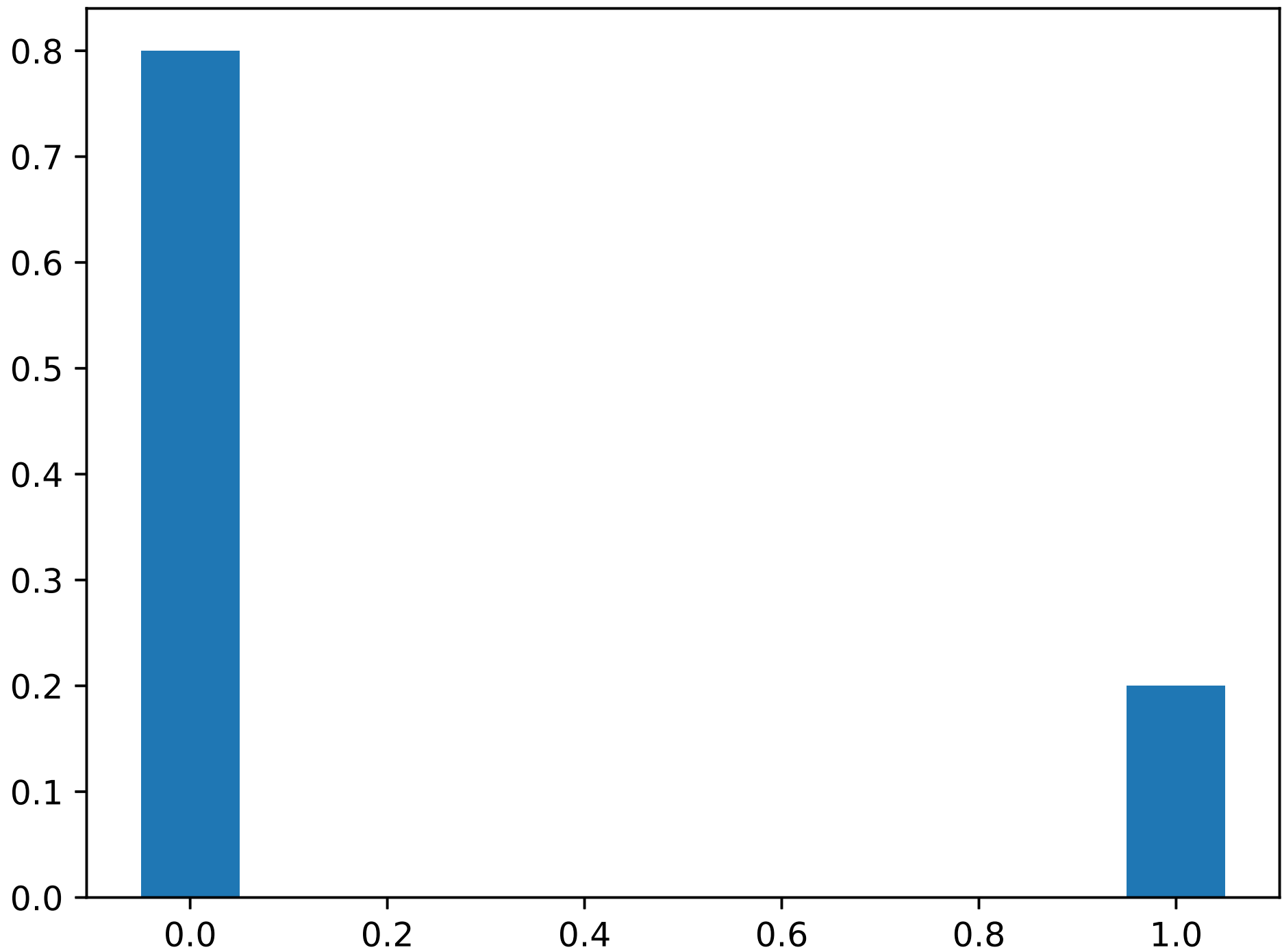
- Supón que tenemos dos cajas con 80 galletas cada una. La primera caja contiene 20 galletas de chocolate y 60 sin chocolate. La segunda caja contiene 40 galletas de cada tipo. Ahora supón que se escoge una caja al azar, y luego una galleta al azar de la caja escogida. Halla la probabilidad de que la galleta escogida proceda de la primera caja sabiendo que es de chocolate.

2.4. Distribuciones Bernoulli y binomial

► Bernoulli:

```
2.4.1.Ber.py
1 from scipy.stats import bernoulli
2 import matplotlib.pyplot as plt
3
4 t = 0.2
5 Y = bernoulli(t)
6 y = [0, 1]
7 print(y, Y.pmf(y), Y.rvs(10))
8 plt.bar(y, Y.pmf(y), width=0.1)
9 plt.show()
```

```
2.4.1.Ber.out
1 [0, 1] [0.8 0.2] [1 0 1 0 0 0 0 1 0 0]
```



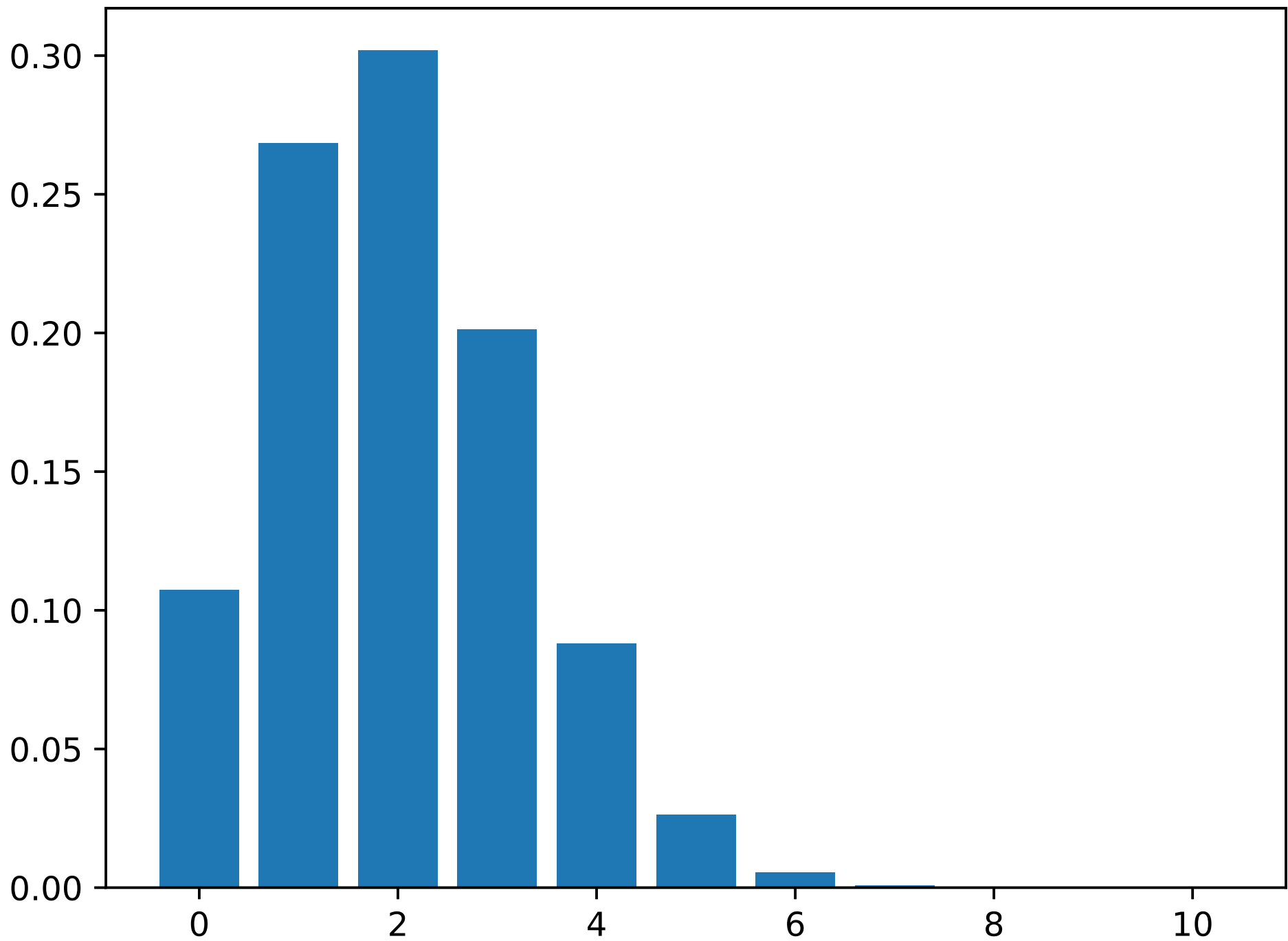
► Binomial:

2.4.1.Bin.py

```
1 import numpy as np
2 from scipy.stats import binom
3 import matplotlib.pyplot as plt
4
5 N, t = 10, 0.2
6 S = binom(N, t)
7 s = np.arange(N+1)
8 print(s, S.pmf(s), S.rvs(10))
9 plt.bar(s, S.pmf(s))
10 plt.show()
```

2.4.1.Bin.out

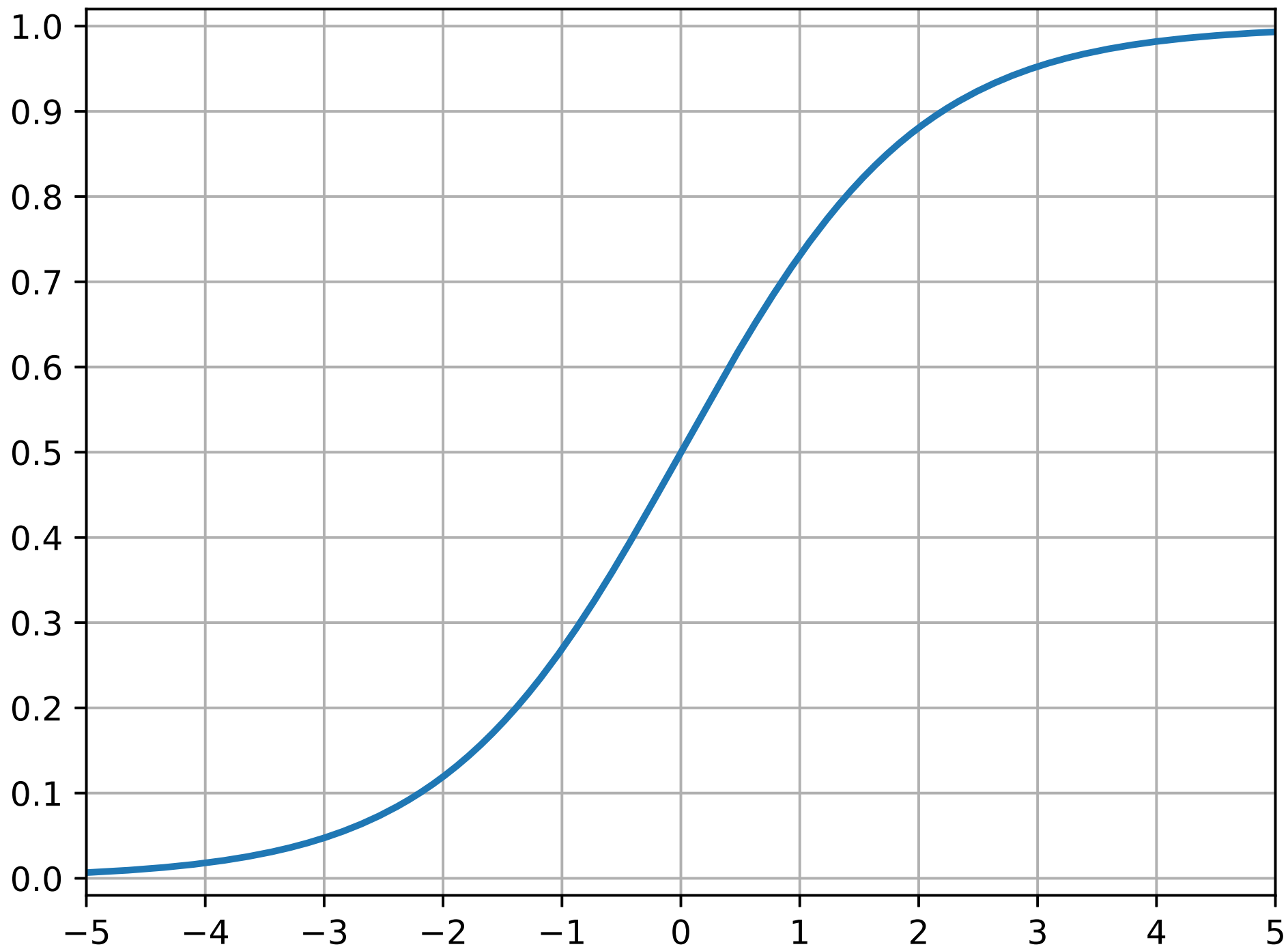
```
1 [ 0  1  2  3  4  5  6  7  8  9 10] [1.07374182e-01
  ↳ 2.68435456e-01 3.01989888e-01 2.01326592e-01
2 8.80803840e-02 2.64241152e-02 5.50502400e-03 7.86432000e-04
3 7.37280000e-05 4.09600000e-06 1.02400000e-07] [3 2 3 0 2 2 2
  ↳ 2 2 0]
```



► Función logística o sigmoide:

2.4.2.sigmoid.py

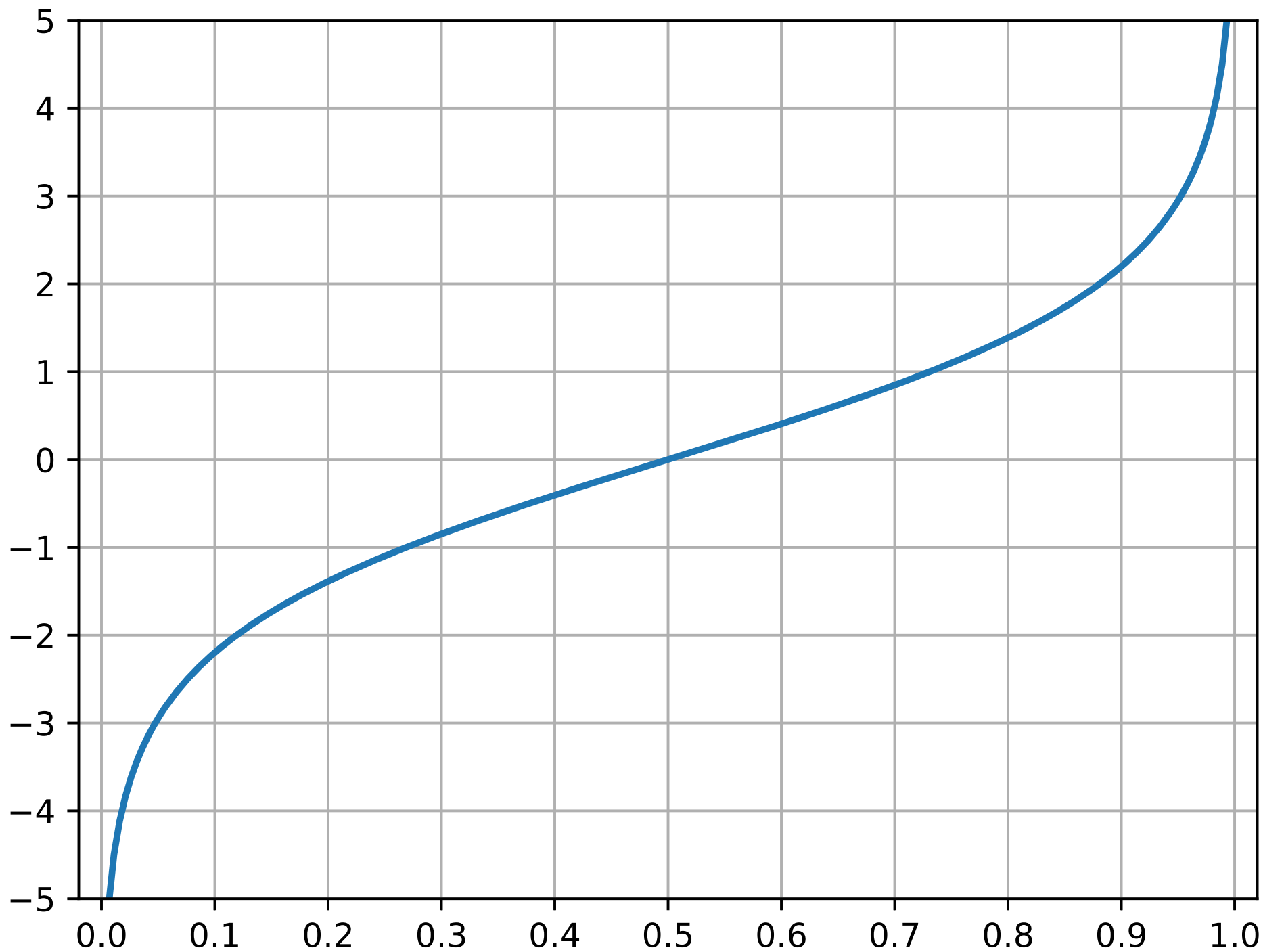
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def sigmoid(a):
5     →return 1 / (1 + np.exp(-a))
6
7 a = np.linspace(-5, 5, 200)
8 plt.figure()
9 plt.plot(a, sigmoid(a), linewidth=2)
10 plt.grid(True)
11 plt.axis([-5, 5, -0.02, 1.02])
12 plt.xticks(np.arange(-5, 6, step=1))
13 plt.yticks(np.arange(0, 1.1, step=0.1))
14 plt.show()
```



► Función logit:

2.4.2.logit.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def logit(p):
5     →return np.log(p / (1 - p))
6
7 p = np.linspace(.001, .999, 200)
8 plt.figure()
9 plt.plot(p, logit(p), linewidth=2)
10 plt.grid(True)
11 plt.axis([-0.02, 1.02, -5, 5])
12 plt.xticks(np.arange(0, 1.1, step=0.1))
13 plt.yticks(np.arange(-5, 6, step=1))
14 plt.show()
```



2.5. Distribuciones categórica y multinomial

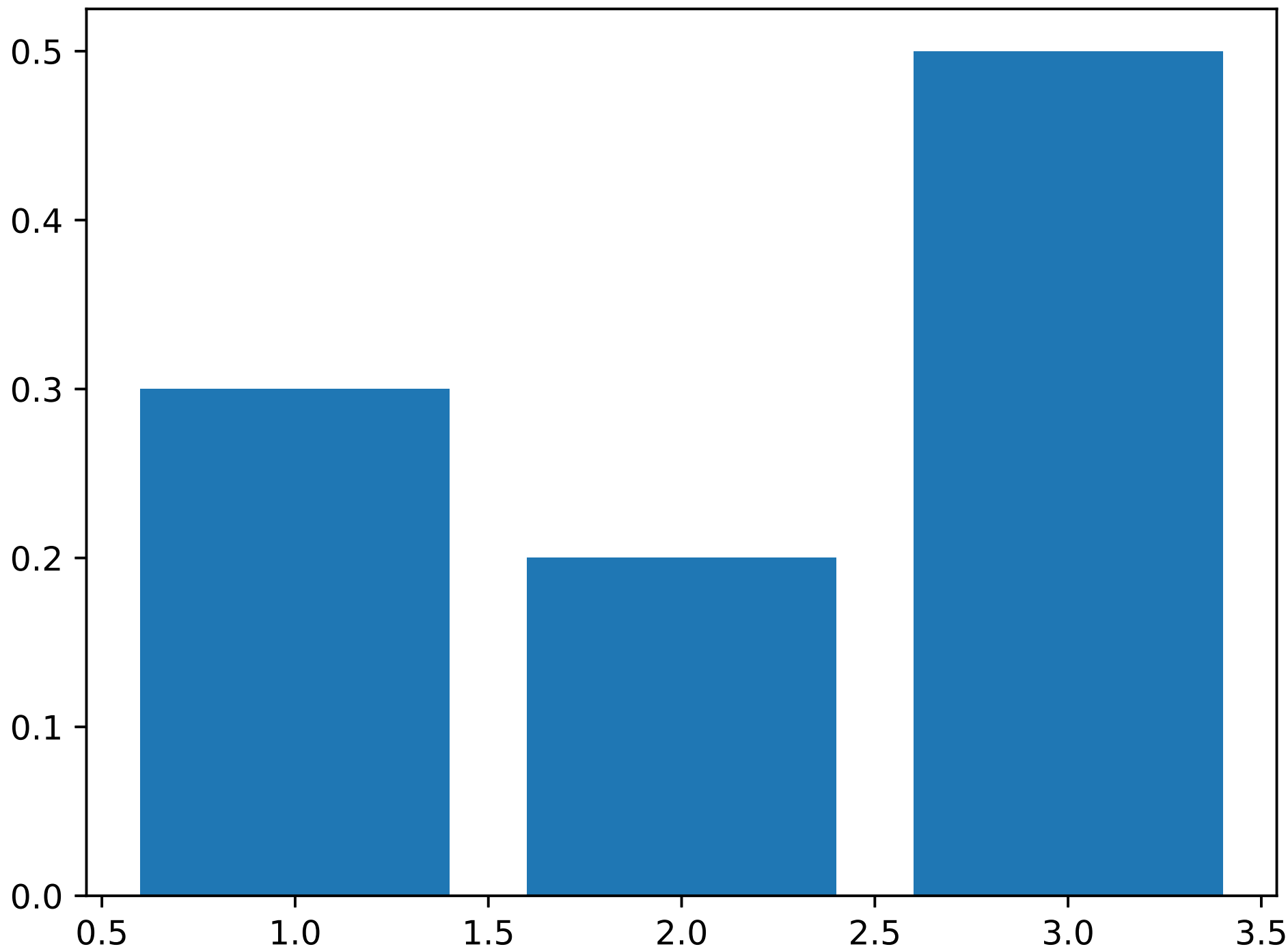
- Multinomial (categórica con $N = 1$):

2.5.1.M.py

```
1 import numpy as np
2 from scipy.stats import multinomial
3 import matplotlib.pyplot as plt
4
5 N, t = 10, [0.3, 0.2, 0.5]
6 S = multinomial(N, t)
7 print(S.pmf([3, 2, 5]))
8 print(S.rvs(4))
9 plt.bar(np.arange(1, len(t)+1), S.p)
10 plt.show()
```

2.5.1.M.out

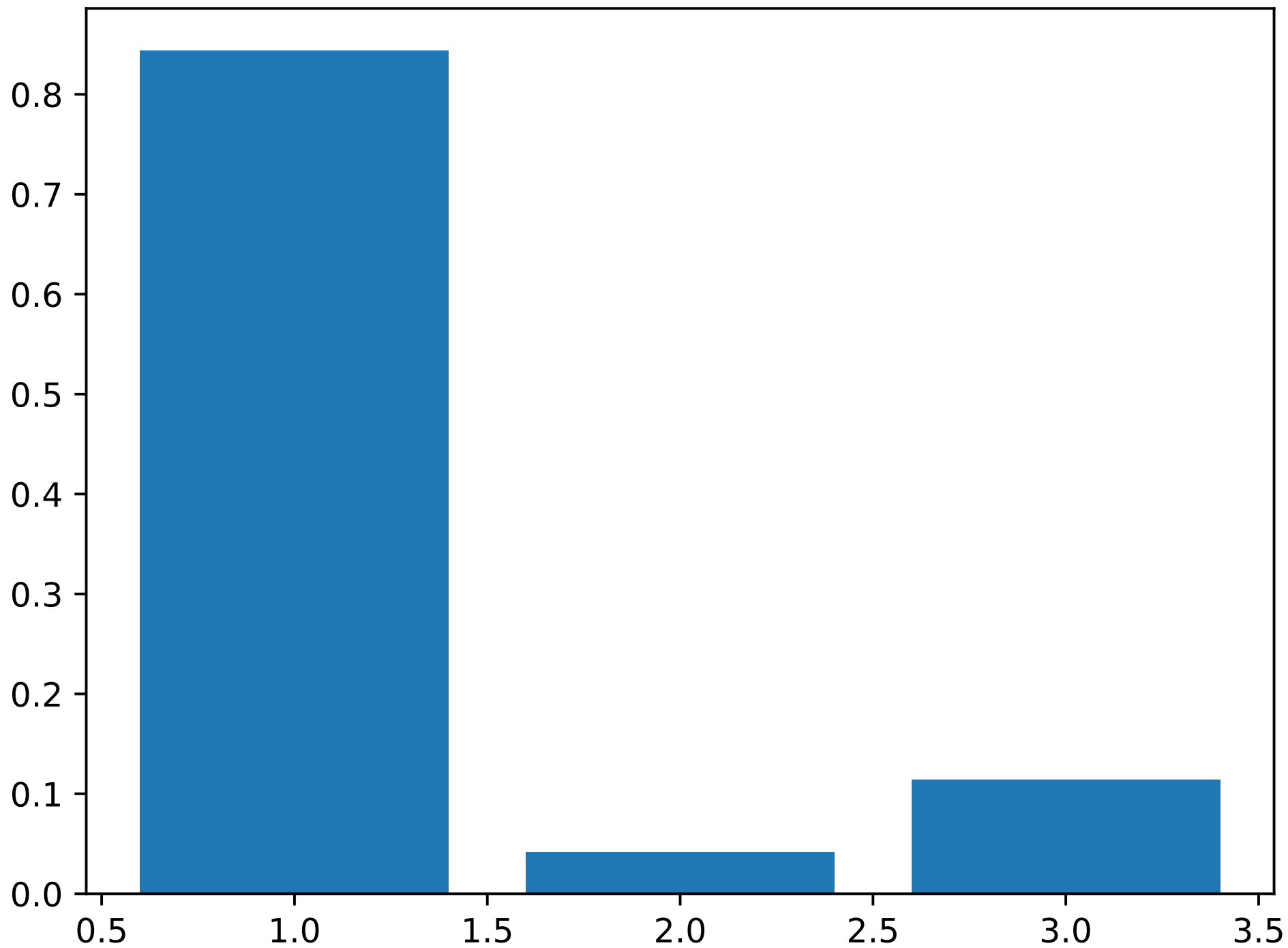
```
1 0.08504999999999999
2 [[1 1 8]
3  [2 4 4]
4  [3 3 4]
5  [4 2 4]]
```



► Softmax:

2.5.2.softmax.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def softmax(a):
5     → e = np.exp((1.0 * np.array(a)))
6     → return e / np.sum(e)
7
8 a = np.array([3, 0, 1])
9 plt.bar(np.arange(1,a.size+1), softmax(a))
10 plt.show()
```



► log-sum-exp:

2.5.4.logsumexp.py

```
1 import numpy as np
2 from scipy.special import logsumexp
3
4 a = 1.0 * np.array([0, 1, 0])
5 print(np.log(np.sum(np.exp(a))), logsumexp(a))
6
7 a = 1.0 * np.array([1000, 1001, 1000])
8 print(np.log(np.sum(np.exp(a))), logsumexp(a))
9
10 a = 1.0 * np.array([-1000, -999, -1000])
11 print(np.log(np.sum(np.exp(a))), logsumexp(a))
```

2.5.4.logsumexp.out

```
1 1.5514447139320509 1.551444713932051
2 inf 1001.551444713932
3 -inf -998.448555286068
```

2.6. Distribución (normal) Gaussiana univariada

► Gaussiana:

2.6.2.norm.py

```
1 import numpy as np
2 from scipy.stats import norm
3 import matplotlib.pyplot as plt
4
5 m, v = 0.0, 1.0
6 Y = norm(m, v)
7 y = np.linspace(-3, 3, 200)
8 plt.plot(y, Y.pdf(y))
9 plt.show()
```